

Capítulo 23

Invocación remota

Al terminar este capítulo, entenderás:

- Qué es un protocolo de petición-respuesta.
- Cuáles son las distintas semánticas de invocación.
- Qué es y cómo funciona la invocación remota.
- Qué son RPC y RMI.

El capítulo 15 presentó el patrón petición-respuesta: el cliente envía una petición —normalmente una operación sobre un recurso— y el servidor responde con un resultado. Ese patrón lo define, en cada caso, un protocolo de aplicación propio: HTTP para la web, SMTP para el correo, DNS para la resolución de nombres, etc. Cada uno especifica su propio formato de mensajes, sus propias operaciones y su propia forma de indicar errores, adaptados al problema concreto que resuelven.

Diseñar un protocolo de aplicación desde cero no es trivial: hay que definir la sintaxis de los mensajes, decidir cómo se delimitan sobre el flujo de bytes subyacente, considerar la evolución del protocolo (versiones, compatibilidad) y, finalmente, implementar esa especificación en cada lenguaje o plataforma que vaya a usarla. Para muchas aplicaciones, ese coste es desproporcionado en relación con lo que realmente necesitan: simplemente ejecutar código en un nodo remoto y obtener el resultado, igual que se invoca una función o un método local.

La **invocación remota** es una solución general para esa necesidad. En lugar de definir un protocolo específico por aplicación, ofrece al programador una infraestructura genérica —el **middleware** de comunicaciones—, común a cualquier aplicación, que se encarga de transportar la invocación hasta el nodo remoto, ejecutarla allí y devolver el resultado. RPC (Remote Procedure Call) sigue este enfoque para procedimientos (funciones independientes); RMI (Remote Method Invocation) hace lo mismo para métodos de objetos. En ambos casos, el middleware genera el código encargado de

establecer la comunicación, serializar los argumentos, enviarlos, ejecutar el código, devolver la respuesta y entregar el resultado de vuelta al cliente, de modo que invocar una operación remota se parece, desde el punto de vista del programador, a invocar una operación local.

Lo que realmente diferencia una invocación remota de un mero intercambio de mensajes es que la invocación implica la ejecución de un fragmento de código en el receptor a expensas del emisor. La petición indica qué pieza de código se ha de ejecutar y con qué argumentos, mientras que la respuesta transporta el resultado de dicha ejecución.

En la invocación remota, el que envía el mensaje de petición (la invocación) es el cliente, y el que recibe y ejecuta el código asociado es el servidor. Sin embargo, aunque es lo habitual, esos roles no tienen porqué coincidir con los de los procesos involucrados, es decir, un servidor TCP podría ser el que envía la invocación hacia el cliente TCP.

23.1. Protocolos de petición-respuesta

Dos procesos que se ejecutan en nodos distintos no comparten memoria, así que la única forma de interactuar es enviándose mensajes a través de la red: eso es el *paso de mensajes*. Se reduce, en su forma más básica, a dos primitivas sobre un socket, `send(destino, mensaje)` y `receive(origen, mensaje)`, sin que deba existir ninguna relación entre mensajes sucesivos.

El patrón *petición-respuesta* es la forma más simple de paso de mensajes en una comunicación extremo a extremo: asocia un mensaje de respuesta al de petición petición que la originó. Este patrón Lo emplean protocolos de varias capas, pero es especialmente frecuente en la de aplicación.

Diseñar un protocolo de aplicación obliga al diseñador a tomar esencialmente las mismas decisiones, independientemente del problema que resuelva el protocolo, aunque el resultado de esas decisiones varía según las necesidades de la aplicación.

La invocación remota no es diferente, aunque no resuelve todas esas decisiones del mismo modo: el middleware fija de antemano algunas de ellas al generar automáticamente el protocolo subyacente, mientras que deja otras en manos del programador. Estas son algunas:

- **Síncrono vs. asíncrono:** el cliente de la petición puede bloquear en espera de la respuesta, o continuar inmediatamente con otra tarea. La primera es una invocación síncrona y es más fácil de entender, porque el cliente sabe en todo momento si la petición sigue pendiente; además es más similar a una llamada local. Una invocación asíncrona

aprovecha el tiempo de espera, pero debe encargarse de correlacionar más adelante cada respuesta con la petición que le corresponda.

- **Con o sin estado:** el servidor puede mantener el estado entre peticiones sucesivas del mismo cliente, o resolver cada una de forma independiente, sin memoria de las anteriores. Un protocolo sin estado es más sencillo de escalar y de recuperar tras un fallo, porque una réplica del servicio puede atender la siguiente petición sin compartir contexto previo con la que atendió la anterior; a cambio, cualquier información que dependa de peticiones anteriores tiene que viajar explícitamente en cada una.
- **Transporte y fiabilidad:** sobre TCP la conexión ya garantiza entrega y orden, pero establecerla tiene un coste que no siempre compensa para un intercambio breve. Sobre UDP no hay tal garantía: si se pierde el mensaje con la petición o el que lleva la respuesta, nada lo detecta ni corrige.

Entre la fiabilidad completa que ofrece TCP y la ausencia total de UDP hay un término medio (o varios): el protocolo de petición-respuesta puede construir sobre UDP solo las medidas de fiabilidad que necesite, sin la sobrecarga que implican las que no necesita. El identificador de mensaje, como el que llevan las consultas DNS, permite cotejar cada respuesta con la petición que la originó, aunque haya varias en curso o el transporte las reordene; ese mismo identificador le permite al servidor detectar y descartar duplicados en lugar de procesarlos de nuevo. Un temporizador permite detectar peticiones o respuestas perdidas. Con ese mismo temporizador se puede añadir la retransmisión, reintentando la petición un número limitado de veces antes de darla por fallida definitivamente. Identificador, temporizador y retransmisión son las piezas básicas de cualquier protocolo ARQ; cada protocolo de petición-respuesta elige cuáles de ellas necesita, y con qué grado de exigencia.

Estos elementos definen tres estilos de intercambio, según qué mensajes intervienen y con qué fin:

- **R (Request):** el cliente envía la petición y no espera ninguna respuesta (no bloqueante). Sirve para peticiones *one-way* habitualmente sobre transporte no confiable. No hay un valor de retorno, confirmación, timeouts ni retransmisiones. También podría ocurrir que hubiera un resultado o confirmación, pero al cliente de la petición no le interese, por lo que no espera ni recoge el mensaje.

- **RR (Request-Reply):** El cliente espera un mensaje de respuesta (bloqueante). La respuesta, además de potencialmente contener un resultado, puede llevar un código de confirmación o de error. Puede utilizar TCP como transporte o bien implementar confiabilidad básica con un ARQ. RR es el estilo más común en protocolos de aplicación.
- **RRA (Request-Reply-Acknowledge):** cuando el servidor detecta una petición duplicada, no basta con descartarla: si ya ejecutó la operación, tiene que reenviar el mismo resultado en lugar de ejecutarla de nuevo, así que necesita conservarlo en una caché llamada *historia*, indexada por el identificador de mensaje, en lugar de descartarlo justo después de enviarlo. La duda es cuánto tiempo mantener cada entrada: sin ninguna señal del cliente, el servidor solo puede adivinarlo con un temporizador propio y un margen de seguridad. Este estilo añade un tercer mensaje con el que el cliente confirma la recepción de la respuesta, y con el que el servidor sabe explícitamente cuándo puede liberar la entrada de la historia. Sin este mensaje, la historia crecería indefinidamente.

Elegir el estilo de intercambio es una de las decisiones que puede tomar el middleware, mientras que otras —como la elección síncrono/asíncrono— son delegadas al programador. Eso no es algo negativo, sino que precisamente le otorga más control en relación a las necesidades y requisitos de la aplicación.

23.2. Medidas de tolerancia a fallos

Frente a la pérdida de un mensaje, la invocación remota puede aplicar tres medidas, que ofrecen fiabilidad creciente:

- **Reintentar la petición:** si el temporizador expira sin recibir respuesta, el cliente retransmite el mismo mensaje, con el mismo identificador. Se aplica un número limitado de repeticiones, que de alcanzarse, será considerado un error. Obviamente implica obligatoriamente un mensaje de respuesta/confirmación. Sin esta medida, el cliente no puede saber si la petición llegó al servidor.
- **Filtrar duplicados**¹: el servidor conserva los identificadores de las peticiones ya atendidas, para reconocer una retransmisión y no repetir la ejecución de la operación que le corresponde. Sin este filtro, cada retransmisión se procesa de nuevo.

¹denominado a menudo «deduplicación»

- **Retransmisión del resultado:** al detectar una petición duplicada, el servidor no se limita a descartarla, sino que reenvía el resultado que ya calculó y tiene guardado, en lugar de dejar al cliente sin respuesta pese a que la operación sí llegó a ejecutarse.

23.3. Operaciones idempotentes

Una operación es **idempotente** cuando ejecutarla varias veces (con los mismos argumentos) produce sobre el estado del servidor el mismo efecto que ejecutarla una sola vez. La idempotencia es una propiedad de la semántica propia de la operación. Es interesante porque estas operaciones no requieren detección de duplicados, lo que simplifica su implementación.

Un estilo RRA necesita cachear la respuesta de cada petición precisamente en el caso de las operaciones no idempotentes, que normalmente se asume que es caso más común.

Algunos ejemplos de operaciones idempotentes podrían ser:

```
set_size(25)
stop()
car.get_color()
get_time()
```

Es interesante destacar que `get_time()` sea idempotente, porque aunque el valor que retorne sea diferente en cada llamada, cumple la condición de no modificar el estado del servidor.

Mientras que los siguientes son no idempotentes:

```
push("item")
counter.inc()
send_mail("user@example.com", "Hello")
```

Este concepto de idempotencia no coincide con el que se utiliza en matemáticas. Aquel está relacionado con el resultado de la aplicación de una función al resultado previo de esa misma función, que de hecho ni siquiera aplica a varios de los ejemplos anteriores porque requiere que la función tenga al menos un argumento.

23.4. Semánticas de invocación

Combinar las tres medidas de tolerancia a fallos —reintentar petición, filtrar duplicados y retransmitir resultado— da lugar a distintas garantías sobre potencialmente cuántas veces se llega a ejecutar la operación remota. Estas garantías se conocen como semánticas de invocación:

- *maybe*: no aplica ninguna medida. Si se pierde la petición o la respuesta, el cliente no tiene forma de saberlo ni de corregirlo. La operación se ejecuta cero o una vez, sin que el cliente pueda distinguir cuál de los dos casos ha ocurrido. Es la semántica que resulta, por ejemplo, de un estilo R (Request) sobre un transporte no confiable. Por ejemplo es útil cuando el cliente envía mensajes actualizando periódicamente el valor de una medida (*p. ej.* la temperatura de una habitación), y no es un problema que se pierda alguno de esos mensajes.
- *at-least-once*: reintenta la petición hasta tener una confirmación, pero no filtra duplicados. Cada retransmisión que llega al servidor se ejecuta de nuevo, así que la operación puede llegar a ejecutarse una o más veces. Garantiza que se ejecuta al menos una vez, a costa de no poder garantizar que no se repita. Se puede implementar con el estilo RR o R con transporte confiable si no requiere valor de retorno.
- *at-most-once*: además de reintentar, el servidor filtra los duplicados. Conserva el resultado ya calculado de la primera ejecución para volver a enviarlo si la petición se repite. La operación se ejecuta como mucho una vez (0 o 1 veces). El cliente siempre recibe una respuesta, ya sea para informar del resultado o de un error, es decir, a diferencia de *maybe* el cliente sabe si la operación se ejecutó o no. Es la semántica que ofrece el estilo RRA (Request-Reply-Acknowledge).
- *exactly-once*: ejecuta la operación exactamente una vez. Esto es algo que las tres medidas de tolerancia a fallos no pueden garantizar por sí solas, porque únicamente actúan sobre la comunicación. La garantía *exactly-once* requiere asegurar la continuidad del estado del servidor, particularmente entre la ejecución de la operación y su registro en la historia, por lo que ambas se ejecutan como una operación atómica. Además requiere de un sistema adicional para recuperar al proceso servidor tras un *crash* con recuperación transparente del estado previo. Eso lo convierte en un problema realmente complejo y difícilmente justificable. En la práctica, lo que se suele implementar es una variante más simple llamada *effectively-once*, que es una combinación de *at-least-once* sobre operaciones idempotentes.

23.5. Mecanismo de invocación remota

Con independencia de la tecnología concreta —RPC, RMI u otra—, todo mecanismo de invocación remota comparte una arquitectura similar. Comienza con una *interfaz remota* conocida tanto por el cliente como por el servidor. La interfaz es una lista de las operaciones que ofrecerá (implemen-

tará) el servidor, con sus nombres, parámetros y tipos de retorno. mientras que el cliente será quien invoque esas operaciones.

El middleware proporciona habitualmente un compilador que genera el código necesario para transportar las invocaciones (identificadores y argumentos) hasta el servidor y llevar los resultados de vuelta al cliente.

Esos fragmentos de código se denominan *stubs* (cabos) y realizan tareas diferentes para cliente y el servidor. Los *stubs* del cliente proporcionan funciones que implementan la interfaz remota, pero no tienen funcionalidad de la aplicación. Lo que hacen es serializar y transportar las invocaciones hasta el servidor. Los *stubs* del servidor incluyen la lógica de multiplexación que localiza y ejecuta funciones con las mismas firmas. Es habitual que ese compilador ofrezca la posibilidad de generar implementaciones huecas para las operaciones, que suelen denominarse *esqueletos* (*skeletons*).

El recorrido completo de una invocación remota, desde que el cliente invoca su *stub* hasta que recibe el resultado, sigue siempre estos pasos:

Cliente	Servidor
El código del cliente invoca la operación remota pasando los argumentos correspondientes, exactamente igual que invocaría una función o un método local, pero en realidad está invocando al <i>stub</i> .	
El <i>stub</i> serializa el identificador de la operación solicitada junto con los argumentos recibidos, y con esa información construye el mensaje de petición.	
El <i>stub</i> del cliente envía la petición al nodo servidor, con el estilo de intercambio y las medidas de fiabilidad aplicados.	
	El <i>stub</i> del servidor recibe la petición. El identificador de operación incluido en el mensaje indica qué procedimiento o método se ha de ejecutar.
	El <i>stub</i> del servidor deserializa los argumentos contenidos en el mensaje, reconstruyendo a partir de ellos los valores nativos que necesita la operación.
	El <i>stub</i> del servidor invoca localmente la implementación real de la operación.
	Cuando la ejecución termina, el <i>stub</i> del servidor serializa el resultado —o la información del error, si la operación falló— en un mensaje de respuesta.
	El <i>runtime</i> del servidor envía la respuesta.
El <i>runtime</i> del cliente la recibe y la entrega al <i>stub</i> .	
El <i>stub</i> deserializa el resultado y lo devuelve como valor de retorno de la llamada original.	

El identificador de la operación (paso 4) le indica al servidor qué operación quiere ejecutar. No se debe confundir con el identificador de mensaje (§23.1), que se utiliza para distinguir ocurrencias distintas, incluso aunque correspondan a la misma operación.

El mecanismo de invocación remota esencialmente ofrece transparencia: la invocación al método remoto es muy similar sintácticamente a una llamada local equivalente. Sin embargo, hay varias diferencias que no se pueden (y no se deben) ocultar: latencia, errores en el servidor, fallo parcial o las garantías de entrega aplicadas (§23.4).

23.5.1. pico-rpc

pico-rpc es un middleware RPC mínimo, escrito en Python con fines exclusivamente docentes. Implementa cada una de las piezas del mecanismo de invocación remota: lenguaje de descripción de interfaces, compilador, generación de los *stubs* de cliente y servidor, y *dispatcher*. Con él resulta sencillo seguir el recorrido completo de una estructura que sigue la descripción de la implementación de RPC de COULOURIS *et al.* [61]. Consta de los siguientes ficheros:

- `picorpc.py`: el *runtime* del middleware. Contiene las primitivas para describir interfaces y el *dispatcher* del servidor.
- `rpcgen.py`: el compilador de interfaces, que genera los *stubs* a partir de la definición de la interfaz.
- `math.idl.py`: la definición de la interfaz del ejemplo.
- `server.py` y `client.py`: es la implementación de los procedimientos remotos, que tendría que aportar el programador de la aplicación.

Interfaz

Normalmente, la definición de la interfaz se escribe con un lenguaje puramente declarativo y específico, llamado IDL, que es independiente del lenguaje de programación en el que se implementa el cliente y el servidor. El compilador genera los *stubs* en el lenguaje de destino a partir de esa definición. Por simplicidad, en pico-rpc, la definición de la interfaz es una clase que hereda de `Interface`, con un método sin cuerpo por cada operación. Las anotaciones de tipo de los parámetros y del valor de retorno determinan cómo se serializa cada valor (Listado 23.1).

```
from picorpc import Interface, uint8, uint64

class Math(Interface):
    def factorial(n: uint8) -> uint64: ...
    def power(base: uint8, exp: uint8) -> uint64: ...
```

LISTADO 23.1: Interfaz del servicio Math
 `/pico-rpc/math.idl.py`

Los tipos `uint8` y `uint64` son simplemente los caracteres de formato de `struct` para enteros sin signo de 1 y 8 bytes ('B' y 'Q'). El identificador de cada operación es su posición en la interfaz: 0 para `factorial()` y 1 para `power()`.

El «compilador» lee esta definición y genera los *stubs* de cliente y servidor:

```
$ ./rpcgen.py math.idl.py
generated math_client_stub.py and math_server_stub.py from math.idl.py
```

Cliente

El *stub* de cliente es una clase (`MathStub`) con un método por cada operación de la interfaz (Listado 23.2). Cada método abre una conexión TCP con el servidor, construye el mensaje de petición con el identificador de la operación (1 byte) seguido de los argumentos serializados, y espera la respuesta, que contiene el resultado serializado como un entero de 8 bytes. Esto encaja con el estilo RR en el que cada conexión transporta una única petición y su respuesta. No hace falta un identificador de mensaje para asociarlas, porque la conexión no se reutiliza y TCP resuelve las retransmisiones de segmentos sin que la aplicación reciba los duplicados.

```
class MathStub:
    FACTORIAL = 0
    POWER = 1

    def __init__(self, host, port):
        self.server_endpoint = host, port

    def factorial(self, n):
        sock = socket.socket()
        sock.connect(self.server_endpoint)

        request = struct.pack('BB', self.FACTORIAL, n)
        sock.sendall(request)

        reply = sock.recv(MAX_MESSAGE_SIZE)
        result = struct.unpack('Q', reply)[0]

        sock.close()
        return result
```

LISTADO 23.2: *Stub* de cliente generado (fragmento)
math_client_stub.py

El programa cliente solo necesita crear el *stub* indicando el *endpoint* del servidor e invocar sus métodos exactamente igual que los de un objeto local (Listado 23.3).

```
import sys

from math_client_stub import MathStub

if len(sys.argv) != 3:
    print("usage: ./client <server> <arg>")
```

```

exit(1)

server = sys.argv[1]
arg = int(sys.argv[2])

stub = MathStub(server, 2000)
print("factorial({}) = {}".format(arg, stub.factorial(arg)))
print("power({}, {}) = {}".format(arg, arg-1, stub.power(arg, arg-1)))

```

LISTADO 23.3: Cliente del servicio Math

[📄/pico-rpc/client.py](#)

Servidor

En el lado del servidor, el compilador genera una función *stub* por cada operación y una tabla (STUBS) que asocia cada identificador de operación con su *stub* (Listado 23.4). Cada *stub* deserializa los argumentos, invoca la implementación real de la operación y serializa el resultado.

```

def factorial_stub(args, implementation):
    args = struct.unpack('B', args)
    result = implementation.factorial(*args)
    return struct.pack('Q', result)

def power_stub(args, implementation):
    args = struct.unpack('BB', args)
    result = implementation.power(*args)
    return struct.pack('Q', result)

FACTORIAL = 0
POWER = 1

STUBS = {
    FACTORIAL: factorial_stub,
    POWER: power_stub,
}

```

LISTADO 23.4: *Stub* de servidor generado[math_server_stub.py](#)

La multiplexación la realiza el *dispatcher* del *runtime* (clase `Dispatcher` de `picorpc.py`). Acepta conexiones de forma iterativa (§ 15.2), lee el mensaje de petición, utiliza su primer byte para localizar el *stub* correspondiente en la tabla STUBS y le entrega el resto del mensaje, es decir, los argumentos serializados. Por último, envía al cliente el resultado que devuelve el *stub*.

El programa servidor define la clase que implementa las operaciones de la interfaz (`MathI`), y registra una instancia en el *dispatcher* junto con el módulo del *stub* de servidor (Listado 23.5).

```

from picorpc import Dispatcher
import math_server_stub

class MathI:
    def factorial(self, n):
        if n == 0:
            return 1

        return n * self.factorial(n - 1)

    def power(self, base, exp):
        return base ** exp

dispatcher = Dispatcher(port=2000)
dispatcher.register(math_server_stub, MathI())
dispatcher.run()

```

LISTADO 23.5: Servidor del servicio Math
 /pico-rpc/server.py

Ejecución del servidor y del cliente en el mismo computador:

Cliente

```

$ ./client.py localhost 5
factorial(5) = '120'
power(5, 4) = '625'

```

Servidor

```

$ ./server.py
Server ready: ('0.0.0.0', 2000)

```

FIGURA 23.1: Servidor y cliente pico-rpc

Para mantener el código al mínimo, pico-rpc prescinde deliberadamente de aspectos que ningún middleware real puede ignorar: solo admite enteros sin signo, supone que cada mensaje llega completo en una única llamada a `recv()` —ignora la E/S parcial—, el servidor es iterativo y no aplica ninguna medida de tolerancia a fallos (ni identificador de mensaje, ni temporizador, ni reintentos). Tampoco transporta errores: si la ejecución de la operación o la serialización de su resultado falla, *p. ej.* `factorial(21)` no cabe en un `uint64`.

23.5.2. ONC RPC

ONC RPC—también llamado Sun RPC— fue uno de los middlewares pioneros de la invocación remota. Lo desarrolló Sun Microsystems en los años 80 y sobre él está construido el popular sistema de ficheros en red NFS. Serializa los datos con el formato XDR [62], y las interfaces de los procedimientos remotos se describen en ficheros `.x`, en un lenguaje que extiende el de XDR que permite declarar programas y procedimientos. La implementación original de Sun, de la que derivan las que se usan hoy, solo permite escribir

clientes y servidores en C, aunque existen implementaciones independientes para otros lenguajes.

El ejemplo `/onc-rpc-factorial` es un servicio equivalente al de `pico-rpc`, con una sola operación. La interfaz (Listado 23.6) declara un programa, identificado por un número (600000002), con una versión (1) que contiene el procedimiento `FACTORIAL`, también numerado (1). El número de procedimiento cumple la misma función que el identificador de operación de `pico-rpc`.

```
program FACTORIAL_PROG {
    version FACTORIAL_VERS {
        unsigned long FACTORIAL(int n) = 1;
    } = 1;
} = 600000002;
```

LISTADO 23.6: Interfaz del servicio `FACTORIAL_PROG`
`/onc-rpc-factorial/factorial.x`

A partir de la interfaz, `rpcgen` (`rpcsvc-proto`) genera tres ficheros: `factorial.h`, con las constantes y los prototipos; `factorial_clnt.c`, el *stub* de cliente, y `factorial_svc.c`, el *stub* de servidor, que incluye el *dispatcher* y la función `main()` del servidor. El programador aporta lo demás:

- `factorial_server.c`: la implementación del procedimiento en la función `factorial_1_svc()`, cuyo nombre se forma con el del procedimiento y el número de versión. Devuelve un puntero a una variable estática porque el *stub* serializa el resultado después de que la función haya retornado (Listado 23.7).
- `factorial_client.c`: obtiene un manejador de cliente con `clnt_create()`, indicando el host, el programa, la versión y el transporte (UDP), e invoca la operación con `factorial_1()`, que es la función del *stub* de cliente (Listado 23.8).

```
u_long *factorial_1_svc(int *n, struct svc_req *rqstp) {
    static u_long result;
    result = 1;

    printf("Received: n=%d \n", *n);

    for (int i=*n; i>0; i--)
        result *= i;

    return &result;
}
```

LISTADO 23.7: Implementación del procedimiento `FACTORIAL`
`/onc-rpc-factorial/factorial_server.c`

```

void factorial_prog_1(char *host, int n) {
    CLIENT *clnt;
    u_long *result_1;

#ifdef DEBUG
    clnt = clnt_create (host, FACTORIAL_PROG, FACTORIAL_VERS, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif

    result_1 = factorial_1(&n, clnt);
    if (result_1 == NULL)
        clnt_perror(clnt, "call failed");
    else
        printf("factorial(%d): %lu", n, *result_1);

#ifdef DEBUG
    clnt_destroy (clnt);
#endif
}

```

LISTADO 23.8: Cliente del servicio FACTORIAL_PROG
 /onc-rpc-factorial/factorial_client.c

Al arrancar, el servidor crea un socket UDP y otro TCP, sin puerto fijo, y los registra en `rpcbind` (versión actual del antiguo «portmapper»), un servicio que asocia cada combinación de programa, versión y transporte con el puerto en el que está disponible. El servidor ofrece los dos transportes para que el cliente pueda elegir cuál usar. Esto se logra con el último argumento de `clnt_create()` (Listado 23.8), y cada uno proporciona una semántica de invocación distinta (§23.4). `clnt_create()` consulta a `rpcbind` en el host remoto para averiguar el puerto que corresponde al transporte elegido.

- Con UDP el *runtime* del cliente retransmite la petición si la respuesta no llega a tiempo, y cada mensaje lleva un identificador de transacción (*xid*) para asociar la respuesta con su petición. Sin embargo, el servidor no filtra duplicados salvo que se active expresamente su caché de peticiones, así que la semántica es *at-least-once*. Con la caché activa, el servidor reenvía la respuesta guardada en lugar de ejecutar de nuevo el procedimiento, lo que se aproxima a *at-most-once*, aunque la caché tiene un tamaño limitado y se pierde si el servidor se reinicia.
- Con TCP el *runtime* del cliente no retransmite. La petición se envía una sola vez y es TCP quien garantiza que llega sin pérdidas ni duplicados. Si la conexión se interrumpe o expira la espera, la llamada devuelve un error sin reintentarlo y el cliente no puede saber si el pro-

cedimiento llegó a ejecutarse. El procedimiento se ejecuta, por tanto, como mucho una vez: semántica es *at-most-once*.

El cliente del ejemplo utiliza UDP. Para este servicio es suficiente, porque calcular el factorial es una operación idempotente.

El ejemplo `Q/onc-rpc-hello` es aún más básico. Solo define un procedimiento que recibe una cadena y la imprime en el servidor. Está tomado, con algunas simplificaciones, de la documentación oficial de ONC RPC [63]. Incluye el mismo programa en dos versiones, una local con una llamada a función convencional (directorio `local`) y otra en la que esa función pasa a ser un procedimiento remoto (directorio `remote`), lo que permite comparar qué cambia al convertir una llamada local en remota.

23.5.3. gRPC

23.6. RMI

23.6.1. ZeroC Ice

23.7. Localización de servicios

23.8. Invocación y despacho asíncrono

23.9. Semántica de fallo

23.10. Rendimiento

23.11. CAP'N Proto

23.12. REST

Y ¿qué más?

El paso de mensajes petición-respuesta no es el único medio sobre el que se construyen abstracciones de comunicación de alto nivel: las colas de mensajes y el modelo de actores, por ejemplo, resuelven necesidades similares con primitivas asíncronas y sin intentar parecerse a una llamada a función.

